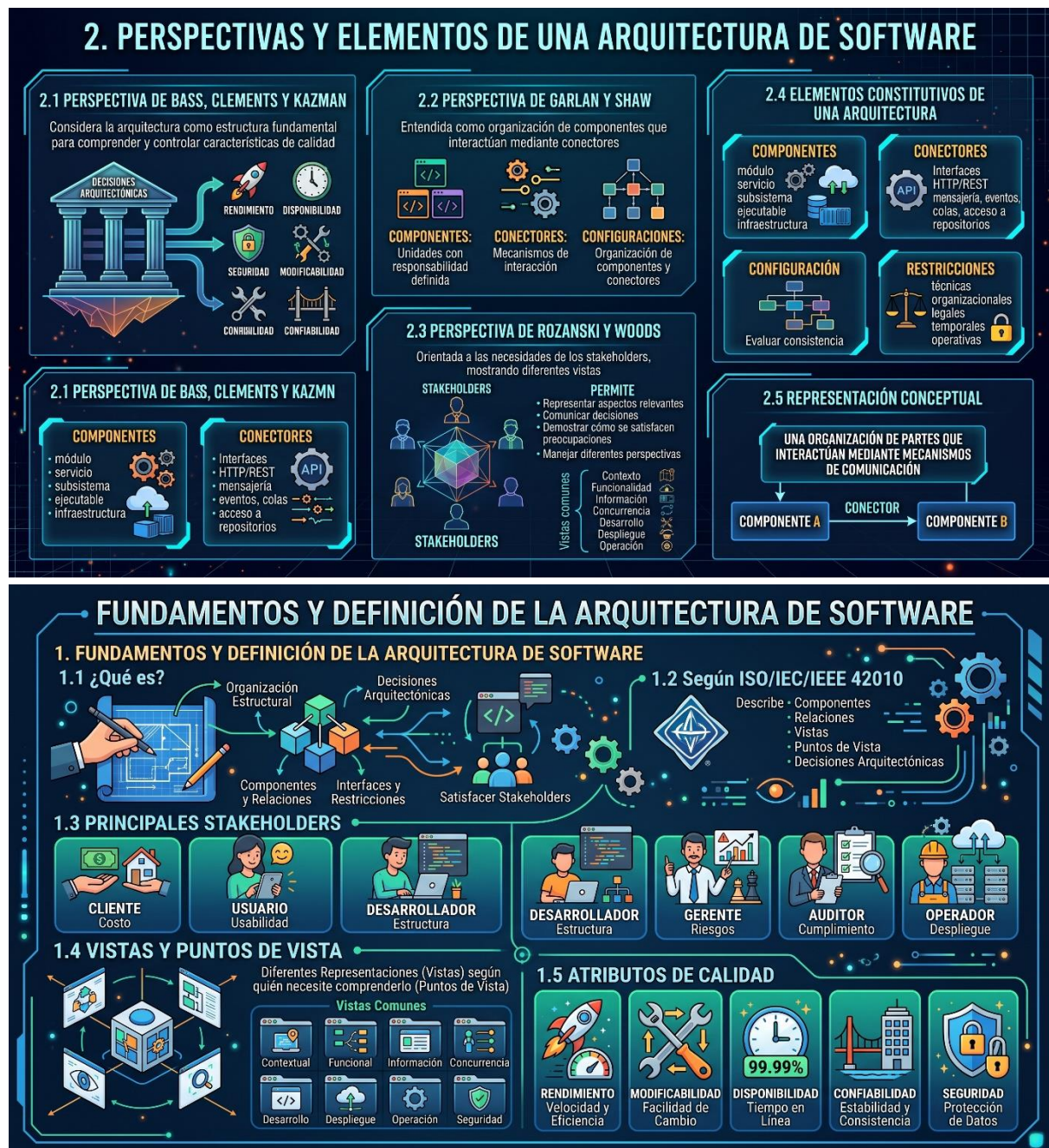


Nombre: Alessander Meza Garay

## Trabajo Semana 1 - Arquitectura de Software





# 3. ARQUITECTURA, DISEÑO, IMPLEMENTACIÓN

## 3.1 Diferencias principales

|                      | Arquitectura<br>alto nivel   | Diseño<br>medio/bajo nivel   | Implementación<br>más bajo nivel     |
|----------------------|------------------------------|------------------------------|--------------------------------------|
| Nivel de abstracción | Altos                        | Medios                       | Bajos                                |
| Alcance              | Sistema completo             | Componentes o clases         | Código ejecutable                    |
| Tipo de decisiones   | Estructurales y estratégicas | Detalladas y técnicas        | Tecnológicas y de desarrollo         |
| Impacto              | Transversal                  | Generalmente localizado      | Se refleja en artefactos de código   |
| Representación       | Vistas arquitectónicas       | UML, clases, secuencias      | Código fuente                        |
| Ejemplo              | Arquitectura por capas       | Patrón Strategy en una clase | Implementación en Java, Python, etc. |

## 3.2 Relación y flujo



## 3.3 Deriva y Erosión arquitectónica

**Deriva arquitectónica**  
Ocurre cuando la implementación se aleja gradualmente de las decisiones arquitectónicas.



**Erosión arquitectónica**  
Degradación más profunda que produce:



## 3.4 Evolución y cambios



# 4. EVOLUCIÓN HISTÓRICA Y ESTILOS ARQUITECTÓNICOS

## 4.1 Evolución histórica



## 4.2 Estilos arquitectónicos principales





## 5. SOA, MICROSERVICIOS Y SISTEMAS DISTRIBUIDOS

Estos enfoques organizan capacidades como servicios y permiten construir sistemas más escalables y flexibles.

### 5.1 (SOA)



#### Principios



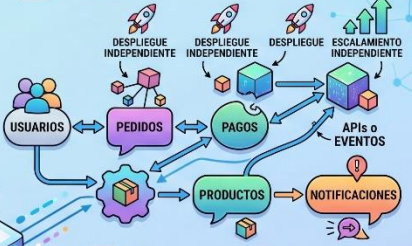
### 5.2 (ESB)



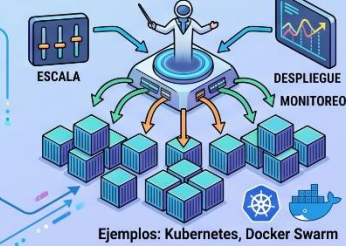
### 5.4 (Contenedores)



### 5.3 (Microservicios)



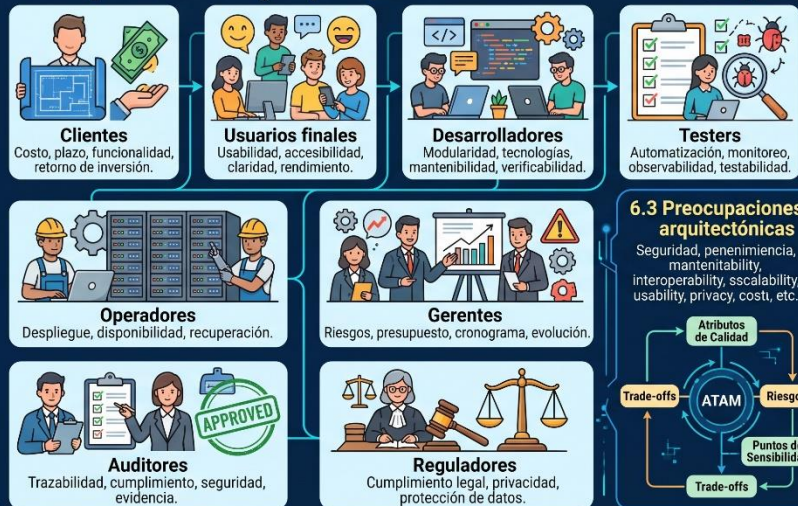
### 5.5 (Orquestación)



## 6. STAKEHOLDERS, PREOCUPACIONES Y ANÁLISIS DE CALIDAD

La arquitectura debe considerar las necesidades y preocupaciones de todos los interesados.

### 6.1 Principales stakeholders y sus preocupaciones



### 6.3 Preocupaciones arquitectónicas

Seguridad, penemencia, mantenibilidad, interoperability, scalability, usability, privacy, cost, etc.



### 6.2 Stakeholders directos e indirectos



### 6.4 ATAM (Architecture Tradeoff Analysis Method)



### 6.5 Escenarios de calidad & 6.6 Utility Tree





## 7. DECISIONES ARQUITECTÓNICAS Y TRADE-OFFS

Una decisión arquitectónica responde a una pregunta importante y tiene impacto en la calidad del sistema.

### 7.1 Naturaleza de las decisiones



### 7.2 Conflictos entre preocupaciones (trade-offs)



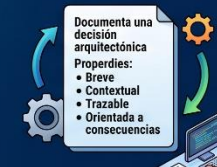
### 7.3 Proceso de toma de decisiones



### 7.4 Costo de cambio



### 7.5 Architecture Decision Record (ADR)



## 8. OBJETIVOS DE LA ARQUITECTURA DE SOFTWARE Y GESTIÓN DE LA COMPLEJIDAD

El objetivo principal es proporcionar una organización que permita comprender y desarrollar el sistema.

### PARTICIONAMIENTO Y MODULARIDAD (8.1, 8.2)



### ABSTRACCIÓN, ENCAPSULAMIENTO Y OCULTAMIENTO (8.5, 8.6, 8.7)



### COHESIÓN Y ACOPLAMIENTO (8.3, 8.4)



### SATISFACER REQUISITOS (8.8)



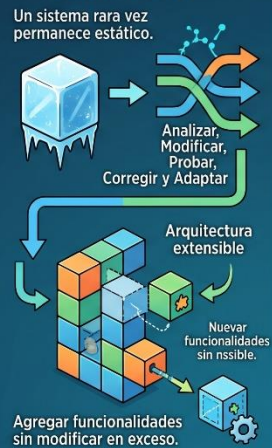
## 9. CALIDAD, EVOLUCIÓN, RIESGOS Y REPRESENTACIÓN ARQUITECTÓNICA

Una buena arquitectura debe asegurar la calidad del sistema, facilitar su evolución y permitir su representación para su análisis y comunicación.

### 9.1 ATRIBUTOS DE CALIDAD (ISO/IEC 25010)



### 9.2 EVOLUCIÓN Y MANTENIMIENTO



### 9.3 RIESGOS Y DEUDA TÉCNICA



### 9.4 ESTILOS ARQUITECTÓNICOS (Resumen)



### 9.5 REPRESENTACIÓN ARQUITECTÓNICA

